SORENTO AI AUTOMATION - FASTAPI BACKEND PROPOSAL

Enterprise REST API with Complete CRUD Operations

Project Scope: Replace Prisma backend with FastAPI for the Sorento Admin Dashboard. Implement RESTful APIs with database models for complete master data management, procurement, inventory, and operational data integration.

Document Version: 1.0
Date: January 14, 2026
Technology Stack: FastAPI, SQLAlchemy ORM, PostgreSQL, Pydantic, Alembic
Location: Kajang, Selangor, MY

EXECUTIVE SUMMARY

This proposal outlines the implementation of a FastAPI-based backend system to replace Prisma. FastAPI provides:

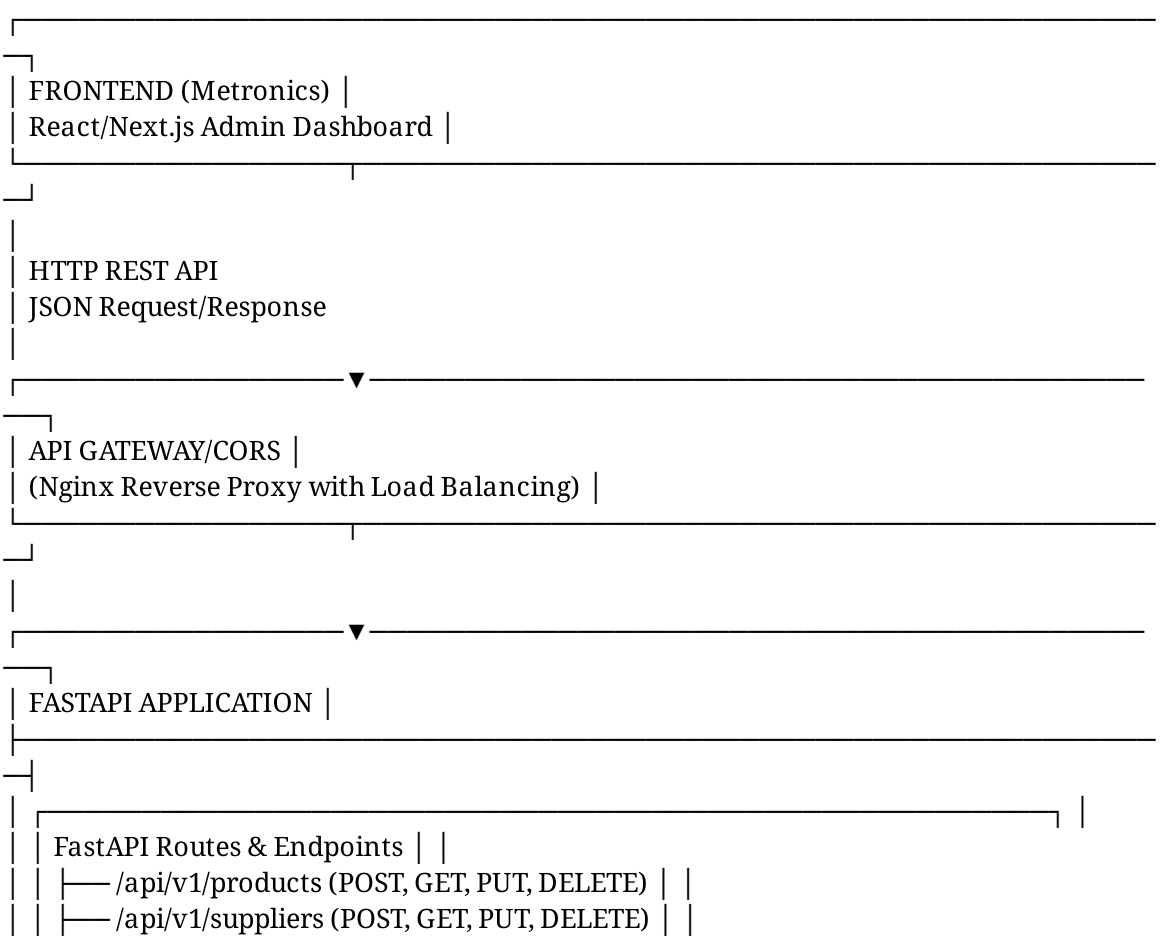
- **High Performance:** Async-first architecture for scalable API endpoints
- **Type Safety:** Pydantic models for automatic validation and serialization
- **Auto-Documentation:** Automatic OpenAPI/Swagger UI generation
- **ORM Integration:** SQLAlchemy for flexible database modeling
- **Production Ready:** Built-in security, CORS, middleware support
- **Easy CRUD:** Simplified database operations with clear patterns

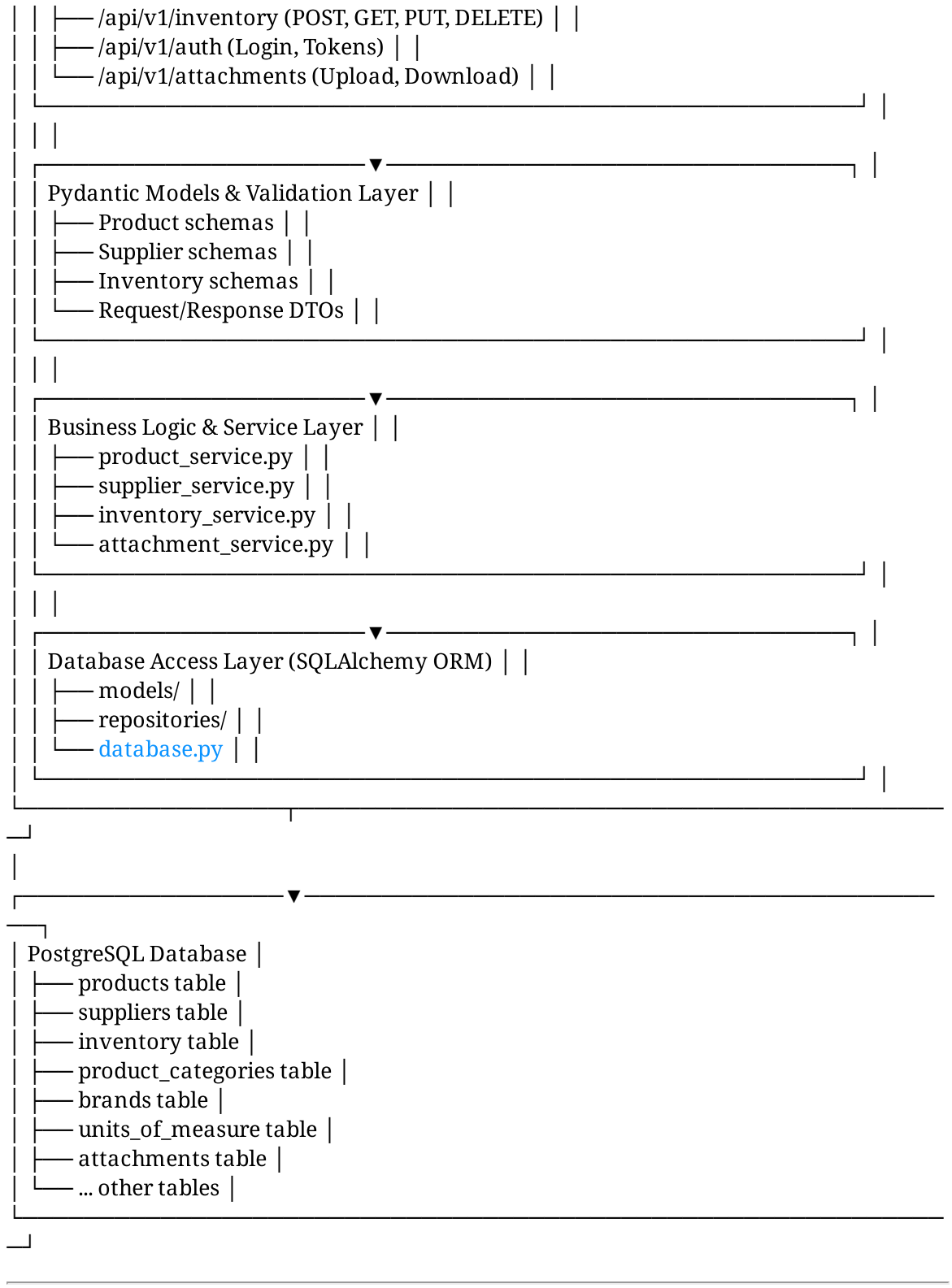
Key Benefits Over Prisma

Aspect	Prisma	FastAPI + SQLAlchemy
Database Language	JavaScript/TypeScript	Python (more data science friendly)
API Framework	GraphQL focused	REST/OpenAPI native
Learning Curve	Schema-first	Code-first with migrations
Performance	Good for GraphQL	Optimized for REST
Async Support	Limited	Native async/await
Deployment	Node.js required	Python ecosystem
Community	Growing	Mature, large Python community

ARCHITECTURE OVERVIEW

High-Level System Design




/api/v1/inventory (POST, GET, PUT, DELETE)
/api/v1/auth (Login, Tokens)
/api/v1/attachments (Upload, Download)
Pydantic Models & Validation Layer
Product schemas
Supplier schemas
Inventory schemas
Request/Response DTOs
Business Logic & Service Layer
product_service.py
supplier_service.py
inventory_service.py
attachment_service.py
Database Access Layer (SQLAlchemy ORM)
models/
repositories/
database.py
PostgreSQL Database
products table
suppliers table
inventory table
product_categories table
brands table
units_of_measure table
attachments table
... other tables

PROJECT STRUCTURE

Recommended Directory Layout

```
sorento-backend/
├── .env # Environment variables (local)
├── .env.example # Environment template
├── .gitignore
├── .dockerignore
├── requirements.txt # Python dependencies
├── pyproject.toml # Project metadata
├── Dockerfile # Production container
├── Dockerfile.dev # Development container
├── docker-compose.yml # Local development
├── docker-compose.prod.yml # Production setup
├── pytest.ini # Testing configuration
├── alembic.ini # Database migrations config

├── app/
│  ├── __init__.py
│  ├── main.py # FastAPI application entry
│  ├── config.py # Configuration management
│  ├── dependencies.py # Dependency injection
│  ├── constants.py # Application constants
│
│ # Database & ORM
│  ├── database.py # Database connection setup
│  ├── models/ # SQLAlchemy ORM models
│  │  ├── __init__.py
│  │  ├── base.py # Base model class
│  │  ├── user.py # User model
│  │  ├── product.py # Product model
│  │  ├── product_category.py # Category model
│  │  ├── brand.py # Brand model
│  │  ├── unit_of_measure.py # UOM model
│  │  ├── supplier.py # Supplier model
│  │  ├── product_supplier.py # Supplier mapping
│  │  ├── inventory.py # Inventory model
│  │  ├── attachment.py # Attachment model
│  │  └── audit_log.py # Audit trail model
│
│ # Request/Response Schemas (Pydantic)
│  ├── schemas/
│  │  ├── __init__.py
│  │  ├── base.py # Base schemas
│  │  ├── product.py # Product request/response DTOs
│  │  ├── product_category.py
│  │  ├── brand.py
│  │  ├── unit_of_measure.py
│  │  ├── supplier.py
│  │  ├── inventory.py
```

```
│   │   ├── attachment.py
│   │   ├── pagination.py # Pagination helpers
│   │   └── error.py # Error response models
│
│   # API Routes
│   ├── api/
│   │   ├── __init__.py
│   │   ├── api_v1.py # API v1 router
│   │   └── v1/
│   │   ├── __init__.py
│   │   ├── auth.py # Authentication endpoints
│   │   ├── products.py # Product CRUD endpoints
│   │   ├── categories.py # Category endpoints
│   │   ├── brands.py # Brand endpoints
│   │   ├── units.py # Unit of measure endpoints
│   │   ├── suppliers.py # Supplier endpoints
│   │   ├── inventory.py # Inventory endpoints
│   │   ├── attachments.py # File upload endpoints
│   │   └── health.py # Health check endpoint
│
│   # Business Logic Layer
│   ├── services/
│   │   ├── __init__.py
│   │   ├── base_service.py # Base service class
│   │   ├── product_service.py
│   │   ├── category_service.py
│   │   ├── brand_service.py
│   │   ├── supplier_service.py
│   │   ├── inventory_service.py
│   │   ├── attachment_service.py
│   │   ├── auth_service.py
│   │   └── file_service.py
│
│   # Database Access Layer
│   ├── repositories/
│   │   ├── __init__.py
│   │   ├── base_repository.py # Generic CRUD operations
│   │   ├── product_repo.py
│   │   ├── category_repo.py
│   │   ├── supplier_repo.py
│   │   ├── inventory_repo.py
│   │   └── attachment_repo.py
│
│   # Utilities
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── security.py # Password hashing, JWT
│   │   ├── validators.py # Custom validators
│   │   ├── formatters.py # Data formatting
│   │   ├── file_utils.py # File operations
│   │   └── logger.py # Logging setup
```

```
│
│ # Middleware & Exceptions
│ ├── middleware/
│ │ ├── __init__.py
│ │ ├── auth_middleware.py # JWT verification
│ │ ├── error_handler.py # Global error handling
│ │ └── request_logger.py # Request/response logging
│
│ # Migrations
│ └── migrations/ # Alembic migrations (auto-generated)

├── tests/
│ ├── __init__.py
│ ├── conftest.py # Pytest fixtures
│ ├── test_products.py # Product endpoint tests
│ ├── test_suppliers.py # Supplier endpoint tests
│ ├── test_inventory.py # Inventory endpoint tests
│ ├── test_auth.py # Authentication tests
│ └── test_services/ # Service layer tests
│ ├── test_product_service.py
│ └── test_supplier_service.py

├── docs/
│ ├── API.md # API documentation
│ ├── SETUP.md # Setup instructions
│ ├── DATABASE.md # Database schema docs
│ └── DEPLOYMENT.md # Deployment guide

└── scripts/
├── init_db.py # Database initialization
├── seed_data.py # Seed sample data
└── generate_migration.sh # Create new migration
```

DATABASE MODELS (SQLAlchemy)

Core Entity Relationships

```
┌──────────────────┐
│ User │ (Authentication)
└──────────────────┘
│
├──────────► Product
│ ├─► ProductCategory
│ ├─► Brand
│ ├─► UnitOfMeasure
│ └─► Attachment
│
├──────────► Supplier
│ ├─► ProductSupplier (N:M with Product)
│ └─► Attachment
│
```

```
├──────► Inventory
│  ├─► Product (FK)
│  └─► Location/Warehouse
│
└──────► AuditLog (Tracks all changes)
```

Model Definitions

File: app/models/user.py

```
from sqlalchemy import Column, String, Boolean, DateTime
from sqlalchemy.orm import relationship
from datetime import datetime
from .base import Base

class User(Base):
__tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String(50), unique=True, index=True, nullable=False)
    email = Column(String(255), unique=True, index=True, nullable=False)
    hashed_password = Column(String(255), nullable=False)
    full_name = Column(String(255), nullable=True)
    is_active = Column(Boolean, default=True)
    is_superuser = Column(Boolean, default=False)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.u

    # Relationships
    products = relationship("Product", back_populates="created_by_user")
    suppliers = relationship("Supplier", back_populates="created_by_user")
    audit_logs = relationship("AuditLog", back_populates="user")
```

File: app/models/product.py

```
from sqlalchemy import Column, Integer, String, Float, Text, DateTime, ForeignKey, Boolean
from sqlalchemy.orm import relationship
from datetime import datetime
from .base import Base

class Product(Base):
__tablename__ = "products"

    id = Column(Integer, primary_key=True, index=True)
    sku = Column(String(100), unique=True, index=True, nullable=False)
```

```
    name = Column(String(255), nullable=False)
    description = Column(Text, nullable=True)
    category_id = Column(Integer, ForeignKey("product_categories.id"), nullable=Fa
    brand_id = Column(Integer, ForeignKey("brands.id"), nullable=True)
    uom_id = Column(Integer, ForeignKey("units_of_measure.id"), nullable=True)

    # Pricing & Status
    cost_price = Column(Float, nullable=False)
    selling_price = Column(Float, nullable=False)
    markup_percentage = Column(Float, nullable=True)
    is_active = Column(Boolean, default=True)
    is_discontinued = Column(Boolean, default=False)

    # Metadata
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.u
    created_by = Column(Integer, ForeignKey("users.id"), nullable=True)

    # Relationships
    category = relationship("ProductCategory", back_populates="products")
    brand = relationship("Brand", back_populates="products")
    uom = relationship("UnitOfMeasure", back_populates="products")
    suppliers = relationship("ProductSupplier", back_populates="product", cascade=
    inventory = relationship("Inventory", back_populates="product", cascade="all, d
    attachments = relationship("Attachment", back_populates="product", cascade="a
    created_by_user = relationship("User", back_populates="products")
```

File: app/models/supplier.py

from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey, Boolean, Text
from sqlalchemy.orm import relationship
from datetime import datetime
from .base import Base

class Supplier(Base):
tablename = "suppliers"

```
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(255), nullable=False)
    code = Column(String(100), unique=True, index=True, nullable=False)
```

```
    contact_person = Column(String(255), nullable=True)
    email = Column(String(255), nullable=True)
    phone = Column(String(20), nullable=True)
    address = Column(Text, nullable=True)
    city = Column(String(100), nullable=True)
    state = Column(String(100), nullable=True)
    country = Column(String(100), nullable=True)
    postal_code = Column(String(10), nullable=True)

    # Business Info
    tax_id = Column(String(50), nullable=True)
    payment_terms = Column(String(100), nullable=True)
    lead_time_days = Column(Integer, default=7)
    minimum_order = Column(Float, nullable=True)
    is_active = Column(Boolean, default=True)
    rating = Column(Float, nullable=True)  # 0-5 rating

    # Timestamps
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.u
    created_by = Column(Integer, ForeignKey("users.id"), nullable=True)

    # Relationships
    products = relationship("ProductSupplier", back_populates="supplier", cascade=
    attachments = relationship("Attachment", back_populates="supplier", cascade="
    created_by_user = relationship("User", back_populates="suppliers")
```

File: `app/models/inventory.py`

```
from sqlalchemy import Column, Integer, Float, DateTime, ForeignKey, String, Text
from sqlalchemy.orm import relationship
from datetime import datetime
from .base import Base

class Inventory(Base):
    __tablename__ = "inventory"

    id = Column(Integer, primary_key=True, index=True)
    product_id = Column(Integer, ForeignKey("products.id"), nullable=False)
    warehouse_location = Column(String(100), nullable=True)
```

```
    quantity_on_hand = Column(Float, default=0.0)
    quantity_reserved = Column(Float, default=0.0)
    quantity_available = Column(Float, default=0.0)  # = on_hand - reserved
    reorder_point = Column(Float, nullable=True)
    reorder_quantity = Column(Float, nullable=True)
    last_counted_at = Column(DateTime, nullable=True)
    last_stock_in = Column(DateTime, nullable=True)
    last_stock_out = Column(DateTime, nullable=True)
    notes = Column(Text, nullable=True)

    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.u

    # Relationships
    product = relationship("Product", back_populates="inventory")
```

File: app/models/attachment.py

```
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey, Float, Enum as SQLEnum
from sqlalchemy.orm import relationship
from datetime import datetime
from enum import Enum
from .base import Base

class AttachmentType(str, Enum):
    PRODUCT_IMAGE = "product_image"
    PRODUCT_SPEC = "product_specification"
    SUPPLIER_DOCUMENT = "supplier_document"
    CERTIFICATE = "certificate"
    INVOICE = "invoice"
    OTHER = "other"

class Attachment(Base):
    __tablename__ = "attachments"

    id = Column(Integer, primary_key=True, index=True)
    filename = Column(String(255), nullable=False)
    original_filename = Column(String(255), nullable=False)
    file_path = Column(String(500), nullable=False)
    file_size = Column(Float, nullable=False)  # in bytes
    mime_type = Column(String(100), nullable=False)
    attachment_type = Column(SQLEnum(AttachmentType), nullable=False)
```

```
    # Foreign keys (polymorphic - can attach to product or supplier)
    product_id = Column(Integer, ForeignKey("products.id"), nullable=True)
    supplier_id = Column(Integer, ForeignKey("suppliers.id"), nullable=True)

    uploaded_at = Column(DateTime, default=datetime.utcnow)
    uploaded_by = Column(Integer, ForeignKey("users.id"), nullable=True)

    # Relationships
    product = relationship("Product", back_populates="attachments")
    supplier = relationship("Supplier", back_populates="attachments")
    uploaded_by_user = relationship("User")
```

Additional Models (ProductCategory, Brand, UnitOfMeasure, ProductSupplier)

app/models/product_category.py

class ProductCategory(Base):
tablename = "product_categories"

```
    id = Column(Integer, primary_key=True)
    name = Column(String(255), unique=True, nullable=False)
    description = Column(Text, nullable=True)
    parent_category_id = Column(Integer, ForeignKey("product_categories.id"), null
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)

    # Relationships
    products = relationship("Product", back_populates="category")
    parent = relationship("ProductCategory", remote_side=[id], backref="children")
```

app/models/brand.py

class Brand(Base):
tablename = "brands"

```
    id = Column(Integer, primary_key=True)
    name = Column(String(255), unique=True, nullable=False)
```

```
    logo_url = Column(String(500), nullable=True)
    description = Column(Text, nullable=True)
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)

    # Relationships
    products = relationship("Product", back_populates="brand")
```

app/models/unit_of_measure.py

class UnitOfMeasure(Base):
tablename = "units_of_measure"

```
    id = Column(Integer, primary_key=True)
    code = Column(String(10), unique=True, nullable=False)
    name = Column(String(100), nullable=False)
    symbol = Column(String(10), nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)

    # Relationships
    products = relationship("Product", back_populates="uom")
```

app/models/product_supplier.py

class ProductSupplier(Base):
tablename = "product_suppliers"

```
    id = Column(Integer, primary_key=True)
    product_id = Column(Integer, ForeignKey("products.id"), nullable=False)
    supplier_id = Column(Integer, ForeignKey("suppliers.id"), nullable=False)
    supplier_sku = Column(String(100), nullable=True)
    supplier_price = Column(Float, nullable=False)
    lead_time_days = Column(Integer, default=7)
    minimum_quantity = Column(Float, default=1.0)
    is_preferred = Column(Boolean, default=False)
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)
```

```
# Relationships
product = relationship("Product", back_populates="suppliers")
supplier = relationship("Supplier", back_populates="products")
```

PYDANTIC SCHEMAS (Request/Response DTOs)

Products API

File: app/schemas/product.py

```
from pydantic import BaseModel, Field, validator
from typing import Optional, List
from datetime import datetime

class ProductBase(BaseModel):
"""Base product schema with common fields"""
sku: str = Field(..., min_length=1, max_length=100)
name: str = Field(..., min_length=1, max_length=255)
description: Optional[str] = None
category_id: int
brand_id: Optional[int] = None
uom_id: Optional[int] = None
cost_price: float = Field(..., gt=0)
selling_price: float = Field(..., gt=0)
markup_percentage: Optional[float] = None

class ProductCreate(ProductBase):
"""Request schema for creating product"""
pass

class ProductUpdate(BaseModel):
"""Request schema for updating product"""
name: Optional[str] = None
description: Optional[str] = None
category_id: Optional[int] = None
brand_id: Optional[int] = None
selling_price: Optional[float] = Field(None, gt=0)
is_active: Optional[bool] = None

class ProductResponse(ProductBase):
"""Response schema for product"""
id: int
is_active: bool
is_discontinued: bool
created_at: datetime
updated_at: datetime
```

```
    class Config:
        from_attributes = True
```

class ProductDetailResponse(ProductResponse):
"""Detailed response including relationships"""
category: 'ProductCategoryResponse'
brand: Optional['BrandResponse']
suppliers: List['ProductSupplierResponse']
inventory: Optional[List['InventoryResponse']]

class PaginatedProductResponse(BaseModel):
"""Paginated product list response"""
total: int
page: int
page_size: int
data: List[ProductResponse]

Suppliers API

File: app/schemas/supplier.py

from pydantic import BaseModel, Field, EmailStr
from typing import Optional, List
from datetime import datetime

class SupplierBase(BaseModel):
name: str = Field(..., min_length=1, max_length=255)
code: str = Field(..., min_length=1, max_length=100)
contact_person: Optional[str] = None
email: Optional[EmailStr] = None
phone: Optional[str] = None
address: Optional[str] = None
city: Optional[str] = None
state: Optional[str] = None
country: Optional[str] = None
postal_code: Optional[str] = None
tax_id: Optional[str] = None
payment_terms: Optional[str] = None
lead_time_days: int = Field(default=7, ge=1)
minimum_order: Optional[float] = None

class SupplierCreate(SupplierBase):
pass

class SupplierUpdate(BaseModel):
name: Optional[str] = None
contact_person: Optional[str] = None
email: Optional[EmailStr] = None
phone: Optional[str] = None
payment_terms: Optional[str] = None

rating: Optional[float] = Field(None, ge=0, le=5)
is_active: Optional[bool] = None

class SupplierResponse(SupplierBase):
id: int
is_active: bool
rating: Optional[float]
created_at: datetime
updated_at: datetime

```
class Config:
    from_attributes = True
```

class SupplierDetailResponse(SupplierResponse):
products: List['ProductSupplierResponse']

Inventory API

File: app/schemas/inventory.py

from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime

class InventoryBase(BaseModel):
product_id: int
warehouse_location: Optional[str] = None
quantity_on_hand: float = Field(default=0.0, ge=0)
quantity_reserved: float = Field(default=0.0, ge=0)
reorder_point: Optional[float] = None
reorder_quantity: Optional[float] = None

class InventoryCreate(InventoryBase):
pass

class InventoryUpdate(BaseModel):
quantity_on_hand: Optional[float] = Field(None, ge=0)
quantity_reserved: Optional[float] = Field(None, ge=0)
reorder_point: Optional[float] = None
warehouse_location: Optional[str] = None

class InventoryResponse(InventoryBase):
id: int
quantity_available: float
last_counted_at: Optional[datetime]
created_at: datetime
updated_at: datetime

```
class Config:
    from_attributes = True
```

API ENDPOINTS (CRUD Operations)

RESTful Endpoint Convention

POST /api/v1/products → Create product
GET /api/v1/products → List products (paginated)
GET /api/v1/products/{id} → Get product detail
PUT /api/v1/products/{id} → Update product
DELETE /api/v1/products/{id} → Delete product

POST /api/v1/suppliers → Create supplier
GET /api/v1/suppliers → List suppliers
GET /api/v1/suppliers/{id} → Get supplier detail
PUT /api/v1/suppliers/{id} → Update supplier
DELETE /api/v1/suppliers/{id} → Delete supplier

POST /api/v1/inventory → Create inventory record
GET /api/v1/inventory → List inventory
GET /api/v1/inventory/{id} → Get inventory detail
PUT /api/v1/inventory/{id} → Update inventory
DELETE /api/v1/inventory/{id} → Delete inventory record

Products API Implementation

File: `app/api/v1/products.py`

from fastapi import APIRouter, Depends, HTTPException, Query, status
from sqlalchemy.orm import Session
from typing import List, Optional
from app.schemas.product import (
ProductCreate, ProductUpdate, ProductResponse, ProductDetailResponse,
PaginatedProductResponse
)
from app.services.product_service import ProductService
from app.database import get_db

router = APIRouter(prefix="/api/v1/products", tags=["products"])

CREATE

@router.post("", response_model=ProductResponse,
status_code=status.HTTP_201_CREATED)
async def create_product(
product_data: ProductCreate,
db: Session = Depends(get_db),
service: ProductService = Depends(lambda db=Depends(get_db): ProductService(db))

):
"""Create new product"""
return service.create(product_data)

READ (List with pagination)

@router.get("", response_model=PaginatedProductResponse)
async def list_products(
page: int = Query(1, ge=1),
page_size: int = Query(10, ge=1, le=100),
category_id: Optional[int] = None,
is_active: Optional[bool] = None,
search: Optional[str] = None,
db: Session = Depends(get_db),
service: ProductService = Depends(lambda db=Depends(get_db): ProductService(db))
):
"""List products with pagination and filtering"""
filters = {
"category_id": category_id,
"is_active": is_active,
"search": search
}
return service.list_paginated(page=page, page_size=page_size, filters=filters)

READ (Single)

@router.get("/{product_id}", response_model=ProductDetailResponse)
async def get_product(
product_id: int,
db: Session = Depends(get_db),
service: ProductService = Depends(lambda db=Depends(get_db): ProductService(db))
):
"""Get product details"""
product = service.get_by_id(product_id)
if not product:
raise HTTPException(status_code=404, detail="Product not found")
return product

UPDATE

@router.put("/{product_id}", response_model=ProductResponse)
async def update_product(
product_id: int,
product_data: ProductUpdate,
db: Session = Depends(get_db),
service: ProductService = Depends(lambda db=Depends(get_db): ProductService(db))
):
"""Update product"""

```
product = service.update(product_id, product_data)
if not product:
raise HTTPException(status_code=404, detail="Product not found")
return product
```

DELETE

```
@router.delete("/{product_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_product(
product_id: int,
db: Session = Depends(get_db),
service: ProductService = Depends(lambda db=Depends(get_db): ProductService(db))
):
"""Delete product"""
success = service.delete(product_id)
if not success:
raise HTTPException(status_code=404, detail="Product not found")
```

Suppliers API Implementation

File: app/api/v1/suppliers.py

```
from fastapi import APIRouter, Depends, HTTPException, Query, status
from sqlalchemy.orm import Session
from typing import List, Optional
from app.schemas.supplier import SupplierCreate, SupplierUpdate, SupplierResponse,
SupplierDetailResponse
from app.services.supplier_service import SupplierService
from app.database import get_db

router = APIRouter(prefix="/api/v1/suppliers", tags=["suppliers"])

@router.post("", response_model=SupplierResponse,
status_code=status.HTTP_201_CREATED)
async def create_supplier(
supplier_data: SupplierCreate,
db: Session = Depends(get_db),
service: SupplierService = Depends(lambda db=Depends(get_db): SupplierService(db))
):
"""Create new supplier"""
return service.create(supplier_data)

@router.get("", response_model=List[SupplierResponse])
async def list_suppliers(
page: int = Query(1, ge=1),
page_size: int = Query(10, ge=1, le=100),
is_active: Optional[bool] = None,
search: Optional[str] = None,
db: Session = Depends(get_db),
service: SupplierService = Depends(lambda db=Depends(get_db): SupplierService(db))
):
```

```
"""List suppliers with filters"""
filters = {"is_active": is_active, "search": search}
return service.list(filters=filters, page=page, page_size=page_size)

@router.get("/{supplier_id}", response_model=SupplierDetailResponse)
async def get_supplier(
supplier_id: int,
db: Session = Depends(get_db),
service: SupplierService = Depends(lambda db=Depends(get_db): SupplierService(db))
):
"""Get supplier details"""
supplier = service.get_by_id(supplier_id)
if not supplier:
raise HTTPException(status_code=404, detail="Supplier not found")
return supplier

@router.put("/{supplier_id}", response_model=SupplierResponse)
async def update_supplier(
supplier_id: int,
supplier_data: SupplierUpdate,
db: Session = Depends(get_db),
service: SupplierService = Depends(lambda db=Depends(get_db): SupplierService(db))
):
"""Update supplier"""
supplier = service.update(supplier_id, supplier_data)
if not supplier:
raise HTTPException(status_code=404, detail="Supplier not found")
return supplier

@router.delete("/{supplier_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_supplier(
supplier_id: int,
db: Session = Depends(get_db),
service: SupplierService = Depends(lambda db=Depends(get_db): SupplierService(db))
):
"""Delete supplier"""
success = service.delete(supplier_id)
if not success:
raise HTTPException(status_code=404, detail="Supplier not found")
```

Inventory API Implementation

File: `app/api/v1/inventory.py`

```
from fastapi import APIRouter, Depends, HTTPException, Query, status
from sqlalchemy.orm import Session
from typing import List, Optional
from app.schemas.inventory import InventoryCreate, InventoryUpdate,
InventoryResponse
from app.services.inventory_service import InventoryService
from app.database import get_db
```

```
router = APIRouter(prefix="/api/v1/inventory", tags=["inventory"])

@router.post("", response_model=InventoryResponse,
status_code=status.HTTP_201_CREATED)
async def create_inventory(
inventory_data: InventoryCreate,
db: Session = Depends(get_db),
service: InventoryService = Depends(lambda db=Depends(get_db): InventoryService(db))
):
"""Create inventory record"""
return service.create(inventory_data)

@router.get("", response_model=List[InventoryResponse])
async def list_inventory(
page: int = Query(1, ge=1),
page_size: int = Query(10, ge=1, le=100),
product_id: Optional[int] = None,
db: Session = Depends(get_db),
service: InventoryService = Depends(lambda db=Depends(get_db): InventoryService(db))
):
"""List inventory records"""
filters = {"product_id": product_id}
return service.list(filters=filters, page=page, page_size=page_size)

@router.get("/{inventory_id}", response_model=InventoryResponse)
async def get_inventory(
inventory_id: int,
db: Session = Depends(get_db),
service: InventoryService = Depends(lambda db=Depends(get_db): InventoryService(db))
):
"""Get inventory details"""
inventory = service.get_by_id(inventory_id)
if not inventory:
raise HTTPException(status_code=404, detail="Inventory record not found")
return inventory

@router.put("/{inventory_id}", response_model=InventoryResponse)
async def update_inventory(
inventory_id: int,
inventory_data: InventoryUpdate,
db: Session = Depends(get_db),
service: InventoryService = Depends(lambda db=Depends(get_db): InventoryService(db))
):
"""Update inventory record"""
inventory = service.update(inventory_id, inventory_data)
if not inventory:
raise HTTPException(status_code=404, detail="Inventory record not found")
return inventory

@router.delete("/{inventory_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_inventory(
inventory_id: int,
```

db: Session = Depends(get_db),
service: InventoryService = Depends(lambda db=Depends(get_db): InventoryService(db))
):
"""Delete inventory record"""
success = service.delete(inventory_id)
if not success:
raise HTTPException(status_code=404, detail="Inventory record not found")

SERVICE LAYER (Business Logic)

Base Service Pattern

File: `app/services/base_service.py`

from sqlalchemy.orm import Session
from sqlalchemy import and_, or_
from typing import Generic, TypeVar, List, Optional, Type, Dict, Any
from pydantic import BaseModel

T = TypeVar('T') # Model type
U = TypeVar('U') # Schema type

class BaseService(Generic[T, U]):
"""Generic service class for CRUD operations"""

```
def __init__(self, db: Session, model: Type[T]):
    self.db = db
    self.model = model

def create(self, obj_in: U) -> T:
    """Create new record"""
    db_obj = self.model(**obj_in.dict(exclude_unset=True))
    self.db.add(db_obj)
    self.db.commit()
    self.db.refresh(db_obj)
    return db_obj

def get_by_id(self, id: int) -> Optional[T]:
    """Get record by ID"""
    return self.db.query(self.model).filter(self.model.id == id).first()

def list(
    self,
    filters: Optional[Dict[str, Any]] = None,
```

```
    page: int = 1,
    page_size: int = 10
) -> List[T]:
    """List records with optional filters and pagination"""
    query = self.db.query(self.model)

    if filters:
        for key, value in filters.items():
            if value is not None and hasattr(self.model, key):
                query = query.filter(getattr(self.model, key) == value)

    skip = (page - 1) * page_size
    return query.offset(skip).limit(page_size).all()

def update(self, id: int, obj_in: U) -> Optional[T]:
    """Update record"""
    db_obj = self.get_by_id(id)
    if not db_obj:
        return None

    update_data = obj_in.dict(exclude_unset=True)
    for field, value in update_data.items():
        setattr(db_obj, field, value)

    self.db.add(db_obj)
    self.db.commit()
    self.db.refresh(db_obj)
    return db_obj

def delete(self, id: int) -> bool:
    """Delete record"""
    db_obj = self.get_by_id(id)
    if not db_obj:
        return False

    self.db.delete(db_obj)
    self.db.commit()
    return True
```

```
def count(self, filters: Optional[Dict[str, Any]] = None) -> int:
    """Count records"""
    query = self.db.query(self.model)

    if filters:
        for key, value in filters.items():
            if value is not None and hasattr(self.model, key):
                query = query.filter(getattr(self.model, key) == value)

    return query.count()

def list_paginated(
    self,
    page: int = 1,
    page_size: int = 10,
    filters: Optional[Dict[str, Any]] = None
) -> Dict[str, Any]:
    """List with pagination info"""
    total = self.count(filters)
    items = self.list(filters=filters, page=page, page_size=page_size)

    return {
        "total": total,
        "page": page,
        "page_size": page_size,
        "data": items
    }
```

Product Service Implementation

File: `app/services/product_service.py`

```
from sqlalchemy.orm import Session
from sqlalchemy import or_
from typing import Optional, Dict, Any
from app.models.product import Product
from app.schemas.product import ProductCreate, ProductUpdate
from .base_service import BaseService
```

class ProductService(BaseService[Product, ProductCreate]):
def **init**(self, db: Session):
super().**init**(db, Product)
self.db = db

```
def create(self, obj_in: ProductCreate) -> Product:
    """Create product with validation"""
    # Check if SKU already exists
    existing = self.db.query(Product).filter(Product.sku == obj_in.sku).first()
    if existing:
        raise ValueError(f"Product with SKU {obj_in.sku} already exists")

    # Calculate markup percentage if not provided
    if not obj_in.markup_percentage:
        markup = ((obj_in.selling_price - obj_in.cost_price) / obj_in.cost_price) * 10
        obj_in.markup_percentage = round(markup, 2)

    return super().create(obj_in)

def list(
    self,
    filters: Optional[Dict[str, Any]] = None,
    page: int = 1,
    page_size: int = 10
) -> list:
    """List products with search functionality"""
    query = self.db.query(Product)

    if filters:
        if filters.get("category_id"):
            query = query.filter(Product.category_id == filters["category_id"])
        if filters.get("is_active") is not None:
            query = query.filter(Product.is_active == filters["is_active"])
        if filters.get("search"):
            search = f"%{filters['search']}%"
            query = query.filter(
                or_(
                    Product.name.ilike(search),
                    Product.sku.ilike(search),
```

```
                Product.description.ilike(search)
            )
        )

    skip = (page - 1) * page_size
    return query.offset(skip).limit(page_size).all()

def get_by_sku(self, sku: str) -> Optional[Product]:
    """Get product by SKU"""
    return self.db.query(Product).filter(Product.sku == sku).first()

def update(self, id: int, obj_in: ProductUpdate) -> Optional[Product]:
    """Update product with recalculation"""
    product = self.get_by_id(id)
    if not product:
        return None

    update_data = obj_in.dict(exclude_unset=True)

    # Recalculate markup if price changed
    if "selling_price" in update_data or "cost_price" in update_data:
        selling = update_data.get("selling_price", product.selling_price)
        cost = update_data.get("cost_price", product.cost_price)
        if cost > 0:
            markup = ((selling - cost) / cost) * 100
            update_data["markup_percentage"] = round(markup, 2)

    for field, value in update_data.items():
        setattr(product, field, value)

    self.db.add(product)
    self.db.commit()
    self.db.refresh(product)
    return product
```

Supplier Service Implementation

File: `app/services/supplier_service.py`

from sqlalchemy.orm import Session
from sqlalchemy import or_
from typing import Optional, Dict, Any
from app.models.supplier import Supplier
from app.schemas.supplier import SupplierCreate, SupplierUpdate
from .base_service import BaseService

class SupplierService(BaseService[Supplier, SupplierCreate]):
def **init**(self, db: Session):
super().**init**(db, Supplier)

```
def create(self, obj_in: SupplierCreate) -> Supplier:
    """Create supplier with validation"""
    # Check if code already exists
    existing = self.db.query(Supplier).filter(Supplier.code == obj_in.code).first()
    if existing:
        raise ValueError(f"Supplier with code {obj_in.code} already exists")

    return super().create(obj_in)

def list(
    self,
    filters: Optional[Dict[str, Any]] = None,
    page: int = 1,
    page_size: int = 10
) -> list:
    """List suppliers with search"""
    query = self.db.query(Supplier)

    if filters:
        if filters.get("is_active") is not None:
            query = query.filter(Supplier.is_active == filters["is_active"])
        if filters.get("search"):
            search = f"%{filters['search']}%"
            query = query.filter(
                or_(
                    Supplier.name.ilike(search),
                    Supplier.code.ilike(search),
```

```
                Supplier.email.ilike(search)
            )
        )

    skip = (page - 1) * page_size
    return query.offset(skip).limit(page_size).all()

def get_by_code(self, code: str) -> Optional[Supplier]:
    """Get supplier by code"""
    return self.db.query(Supplier).filter(Supplier.code == code).first()
```

AUTHENTICATION & SECURITY

JWT Token Setup

File: `app/utils/security.py`

```
from datetime import datetime, timedelta
from typing import Optional
from jose import JWTError, jwt
from passlib.context import CryptContext
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthCredentials

SECRET_KEY = "your-secret-key-change-in-production"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
security = HTTPBearer()

def hash_password(password: str) -> str:
"""Hash password"""
return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
"""Verify password"""
return pwd_context.verify(plain_password, hashed_password)

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
"""Create JWT token"""
to_encode = data.copy()
if expires_delta:
expire = datetime.utcnow() + expires_delta
else:
expire = datetime.utcnow() + timedelta(minutes=15)
```

```
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt
```

```
def verify_token(credentials: HTTPAuthCredentials = Depends(security)) -> str:
"""Verify JWT token"""
token = credentials.credentials
try:
payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
user_id: str = payload.get("sub")
if user_id is None:
raise HTTPException(
status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid token"
)
except JWTError:
raise HTTPException(
status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid token"
)
return user_id
```

Authentication Endpoints

File: `app/api/v1/auth.py`

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from app.database import get_db
from app.models.user import User
from app.utils.security import hash_password, verify_password, create_access_token
from pydantic import BaseModel

router = APIRouter(prefix="/api/v1/auth", tags=["auth"])

class LoginRequest(BaseModel):
username: str
password: str

class TokenResponse(BaseModel):
access_token: str
token_type: str

@router.post("/login", response_model=TokenResponse)
async def login(
credentials: LoginRequest,
db: Session = Depends(get_db)
):
"""Login user and return JWT token"""
user = db.query(User).filter(User.username == credentials.username).first()
```

```
if not user or not verify_password(credentials.password, user.hashed_passwor
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid credentials"
    )

if not user.is_active:
    raise HTTPException(
        status_code=status.HTTP_403_FORBIDDEN,
        detail="User is inactive"
    )

access_token = create_access_token(data={"sub": str(user.id)})
return {"access_token": access_token, "token_type": "bearer"}
```

```
@router.post("/register")
async def register(
credentials: LoginRequest,
db: Session = Depends(get_db)
):
"""Register new user"""
# Check if user exists
existing_user = db.query(User).filter(User.username == credentials.username).first()
if existing_user:
raise HTTPException(
status_code=status.HTTP_400_BAD_REQUEST,
detail="Username already exists"
)
```

```
# Create new user
new_user = User(
    username=credentials.username,
    email=credentials.username,  # Use username as email for now
    hashed_password=hash_password(credentials.password),
    is_active=True
)

db.add(new_user)
db.commit()
db.refresh(new_user)
```

```
return {"id": new_user.id, "username": new_user.username}
```

DOCKER DEPLOYMENT

Dockerfile (Production)

File: `Dockerfile`

```
==========================================
==========================================
```

Stage 1: Builder

```
==========================================
==========================================
```

```
FROM python:3.11-slim as builder

WORKDIR /app

RUN apt-get update && apt-get install -y
gcc
postgresql-client
&& rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt
```

```
==========================================
==========================================
```

Stage 2: Runtime

==========

==========

FROM python:3.11-slim

WORKDIR /app

Install runtime dependencies

RUN apt-get update && apt-get install -y
postgresql-client
&& rm -rf /var/lib/apt/lists/*

Create non-root user

RUN useradd -m -u 1001 appuser

Copy Python dependencies from builder

COPY --from=builder /root/.local /home/appuser/.local

Copy application code

COPY --chown=appuser:appuser . .

Set environment variables

ENV PATH=/home/appuser/.local/bin:$PATH
PYTHONUNBUFFERED=1
PYTHONDONTWRITEBYTECODE=1

USER appuser

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3
CMD python -c "import requests; requests.get('http://localhost:8000/health')" || exit 1

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

Docker Compose

File: `docker-compose.yml` (Development)

version: '3.8'

```
services:
backend:
build:
context: .
dockerfile: Dockerfile.dev
container_name: sorento-backend-dev
ports:
- "8000:8000"
environment:
- DATABASE_URL=postgresql://sorento:sorento_password@postgres:5432/sorento_db
- ENVIRONMENT=development
- DEBUG=true
volumes:
- .:/app
depends_on:
- postgres
command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

postgres:
image: postgres:15-alpine
container_name: sorento-postgres-dev
environment:
- POSTGRES_USER=sorento
- POSTGRES_PASSWORD=sorento_password
- POSTGRES_DB=sorento_db
volumes:
- postgres_data:/var/lib/postgresql/data
ports:
- "5432:5432"

volumes:
postgres_data:
```

FastAPI Main Application

File: `app/main.py`

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.trustedhost import TrustedHostMiddleware
from app.api.api_v1 import api_router
from app.database import engine, Base
from app.config import settings

# Create tables

Base.metadata.create_all(bind=engine)
```

Initialize FastAPI app

app = FastAPI(
title="Sorento API",
description="Backend API for Sorento Admin Dashboard",
version="1.0.0",
docs_url="/api/docs",
openapi_url="/api/openapi.json"
)

Add CORS middleware

app.add_middleware(
CORSMiddleware,
allow_origins=settings.CORS_ORIGINS,
allow_credentials=True,
allow_methods=[""*],*
allow_headers=[""],
)

Add trusted host middleware

app.add_middleware(
TrustedHostMiddleware,
allowed_hosts=settings.ALLOWED_HOSTS
)

Include API routes

app.include_router(api_router)

Health check endpoint

@app.get("/health")
async def health_check():
return {"status": "healthy"}

@app.get("/")
async def root():
return {
"message": "Sorento API",
"version": "1.0.0",
"docs": "/api/docs"
}

IMPLEMENTATION CHECKLIST

Phase 1: Setup & Configuration

- [ ] Create FastAPI project structure
- [ ] Install dependencies (FastAPI, SQLAlchemy, Pydantic, etc.)
- [ ] Configure PostgreSQL connection
- [ ] Setup environment variables (.env files)
- [ ] Initialize Alembic for database migrations

Phase 2: Database Models

- [ ] Create SQLAlchemy models for all entities
- [ ] Define relationships and foreign keys
- [ ] Create base model class with common fields
- [ ] Write initial Alembic migration

Phase 3: Schemas & Validation

- [ ] Create Pydantic schemas for request/response
- [ ] Implement custom validators
- [ ] Define pagination schemas
- [ ] Create error response schemas

Phase 4: API Endpoints (CRUD)

- [ ] Implement Product endpoints (POST, GET, GET/:id, PUT, DELETE)
- [ ] Implement Supplier endpoints
- [ ] Implement Inventory endpoints
- [ ] Implement Category endpoints
- [ ] Implement Brand endpoints
- [ ] Implement Unit of Measure endpoints
- [ ] Implement Product-Supplier mapping endpoints

Phase 5: Business Logic

- [ ] Create base service class
- [ ] Implement ProductService with custom logic
- [ ] Implement SupplierService
- [ ] Implement InventoryService
- [ ] Add search/filter logic to services

Phase 6: Authentication

- [ ] Implement JWT token generation
- [ ] Create login/register endpoints
- [ ] Add authentication middleware
- [ ] Protect endpoints with auth dependencies
- [ ] Implement role-based access control (optional)

Phase 7: File Handling

- [ ] Implement file upload endpoints
- [ ] Setup file storage (local or S3)
- [ ] Create attachment service
- [ ] Link attachments to products/suppliers

Phase 8: Testing

- [ ] Write unit tests for services
- [ ] Write integration tests for endpoints
- [ ] Create test fixtures and factories
- [ ] Achieve minimum 80% code coverage

Phase 9: Documentation

- [ ] Add docstrings to all functions
- [ ] Generate OpenAPI/Swagger documentation
- [ ] Create API documentation guide
- [ ] Document deployment procedures

Phase 10: Deployment

- [ ] Create production Dockerfile
- [ ] Setup Docker Compose for production
- [ ] Configure Nginx reverse proxy
- [ ] Setup SSL certificates
- [ ] Configure environment variables for production
- [ ] Setup database backups

Phase 11: Frontend Integration

- [ ] Update Metronics API calls to use FastAPI endpoints
- [ ] Test CORS configuration
- [ ] Verify all CRUD operations work end-to-end
- [ ] Setup API base URL configuration

Phase 12: Monitoring & Logging

- [ ] Setup application logging
- [ ] Add request/response logging middleware
- [ ] Configure error tracking (Sentry optional)
- [ ] Setup health check monitoring
- [ ] Create uptime monitoring

QUICK START COMMANDS

Create virtual environment

python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

Install dependencies

pip install -r requirements.txt

Initialize database

alembic upgrade head

Run development server

uvicorn app.main:app --reload

Run tests

pytest

Build Docker image

docker build -t sorento-backend:latest .

Run with Docker Compose

docker-compose up -d

View API documentation

http://localhost:8000/api/docs

REFERENCES & RESOURCES

- **FastAPI Documentation:** https://fastapi.tiangolo.com/
- **SQLAlchemy ORM:** https://docs.sqlalchemy.org/
- **Pydantic Validation:** https://docs.pydantic.dev/
- **PostgreSQL:** https://www.postgresql.org/
- **Alembic Migrations:** https://alembic.sqlalchemy.org/
- **JWT Authentication:** https://python-jose.readthedocs.io/
- **Uvicorn Server:** https://www.uvicorn.org/

NEXT STEPS FOR CURSOR

1. **Initialize Project Structure** - Create the directory layout as specified
2. **Install Dependencies** - Run `pip install -r requirements.txt`
3. **Implement Models** - Start with SQLAlchemy models
4. **Create Migrations** - Use Alembic to generate database schema
5. **Implement Services** - Build business logic layer
6. **Create Endpoints** - Implement all CRUD routes
7. **Test Locally** - Verify endpoints with Postman/API client
8. **Dockerize** - Build and test Docker containers
9. **Integrate with Frontend** - Update Metronics to call FastAPI endpoints
10. **Deploy** - Move to production server

MIGRATION FROM PRISMA

Key Differences

Aspect	Prisma	FastAPI
Schema Definition	`schema.prisma` file	Python model classes
Type Generation	Auto-generated types	Pydantic schemas
Migrations	Prisma migrate	Alembic
API Layer	Separate (typically Next.js API routes)	FastAPI routes directly
ORM Queries	Prisma Client	SQLAlchemy Query API
Async Support	Limited	Native async/await

Migration Path

1. Keep Prisma until FastAPI is fully tested
2. Run both backends in parallel (Metronics → FastAPI, no breaking changes)
3. Migrate database: Export Prisma schema → Convert to SQLAlchemy models
4. Gradually replace API endpoints
5. Decommission Prisma once all endpoints working

Status: FastAPI Backend Proposal Complete
Last Updated: January 14, 2026
For: Sorento Admin Dashboard
Technology: FastAPI + SQLAlchemy + PostgreSQL

Document Ready for Cursor Implementation
This comprehensive proposal covers complete CRUD operations for all master data entities with production-ready architecture, security, and deployment strategy.